

Technical Proposal and Research Roadmap

Adaptive Lag-Selective Shrinkage for Robust Coarray MUSIC in Weight-Constrained Sparse Arrays

From Reproducible DOA Experiments to FPGA-Ready Acceleration

Hossein Molhem

Department of Electrical and Computer Engineering
Kennesaw State University
hmolhem@students.kennesaw.edu

May 10, 2026

Abstract

This document defines the technical proposal, research alignment map, repository restructuring plan, and long-term development roadmap for the MIMO_GEOMETRY_ANALYSIS project. The project extends weight-constrained sparse-array direction-of-arrival (DOA) estimation by adding a post-geometry statistical denoising layer for finite-snapshot coarray correlations. The reference work by Kulkarni and Vaidyanathan introduced weight-constrained sparse arrays that reduce mutual coupling sensitivity through geometric constraints such as $w(1) = 0$ and $w(1) = w(2) = 0$. This project complements that geometry-level bias mitigation with Adaptive Lag-Selective Shrinkage (ALSS), a lag-aware regularization method designed to reduce variance in low-confidence coarray lag estimates before virtual covariance reconstruction and Coarray MUSIC processing.

The near-term objective is to rebuild the repository into a coherent, reproducible research framework with exact reference geometries, aligned algorithm definitions, script-generated results, a paper-to-code map, and a clean IEEE-style paper. The long-term objective is to connect the Python research framework to FPGA/HLS acceleration by identifying hardware-friendly kernels such as sample covariance formation, coarray lag averaging, Toeplitz covariance reconstruction, and MUSIC spectrum scanning. This roadmap is intended to serve as the master source document for academic development, GitHub presentation, course projects, and resume positioning.

Contents

1	Project Vision	4
2	Reference Work and Academic Positioning	4
2.1	Reference Paper: Weight-Constrained Sparse Arrays	4
2.2	Identified Gap in the Reference Work	5
2.3	Correct Academic Positioning of This Project	5
3	Research Question and Hypothesis	6
3.1	Main Research Question	6
3.2	Technical Hypothesis	6

3.3	Bias–Variance Interpretation	6
4	Proposed Technical Framework	6
4.1	Layer 1: Geometry and Coarray Analysis	6
4.2	Layer 2: Signal Processing and DOA Estimation	7
4.3	Layer 3: FPGA-Ready Acceleration Path	7
5	ALSS Method Definition	8
5.1	Purpose of ALSS	8
5.2	General Shrinkage Form	8
5.3	Variance Proxy	8
5.3.1	Option A: Retention-Factor ALSS	8
5.3.2	Option B: Variance-Ratio Shrinkage	8
5.4	Core-Lag Protection	9
5.5	Hermitian Symmetry	9
6	Repository Restructuring Plan	9
6.1	Target Repository Structure	9
6.2	Important Naming Rule	10
7	Work Packages	10
7.1	WP1: Literature and Source Alignment	10
7.2	WP2: Geometry Processor Cleanup	11
7.3	WP3: ALSS Algorithm Alignment	11
7.4	WP4: Reproducible Experiments	11
7.5	WP5: Paper-to-Code Map	12
7.6	WP6: IEEE-Style Paper Rewrite	12
7.7	WP7: GitHub and Portfolio Presentation	13
8	Experiment Design	13
8.1	Baseline Signal Model	13
8.2	Mutual Coupling Model	14
8.3	Performance Metrics	14
8.4	Reproducibility Requirements	14
9	FPGA and Embedded-AI Development Path	15
9.1	Connection to the Existing SCM HLS Project	15
9.2	Candidate Hardware Kernels	15
9.3	Recommended Hardware Strategy	15
10	Course and Career Alignment	16
10.1	Summer Course: Embedded AI / Autonomous Robots	16
10.2	Summer Course: Antenna Engineering	16
10.3	Fall Course: Machine Learning	16
10.4	Fall Course: Python for Data Science	17
11	Risk Register and Corrections	17

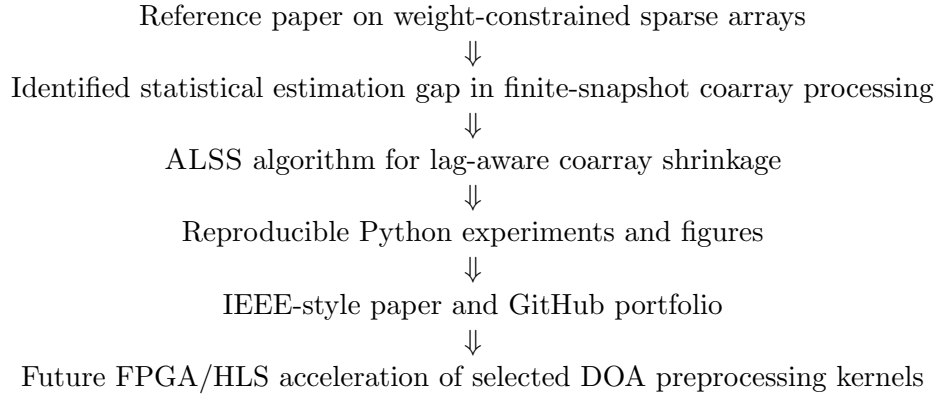
12 Immediate Action Plan	17
12.1 Priority 1: Freeze the Scientific Narrative	17
12.2 Priority 2: Build the Paper-to-Code Map	18
12.3 Priority 3: Correct Geometry Implementations	18
12.4 Priority 4: Regenerate All Results	18
12.5 Priority 5: Rewrite README and Paper	18
13 Expected Final Deliverables	18
14 Resume and Portfolio Positioning	19
15 Conclusion	19

1 Project Vision

The objective of this project is to build a professional research and engineering portfolio around the following technical identity:

**Sparse-array DOA estimation + geometry-aware coarray processing +
statistical denoising + FPGA/HLS acceleration path**

The project should not be treated as a loose collection of Python scripts. It should be treated as a reproducible research platform with a clear academic chain:



The final repository should be understandable by three audiences:

1. **Academic reviewer:** Can verify that the research question, method, experiments, and claims are scientifically consistent.
2. **Professor/course evaluator:** Can see clear links to Antenna Engineering, Embedded AI/Autonomous Robots, Machine Learning, and Python/Data Science coursework.
3. **Hiring manager or technical recruiter:** Can see evidence of practical skill in DSP, radar signal processing, scientific Python, reproducibility, and FPGA/HLS acceleration planning.

2 Reference Work and Academic Positioning

2.1 Reference Paper: Weight-Constrained Sparse Arrays

The core reference paper is:

P. Kulkarni and P. P. Vaidyanathan, “Weight-Constrained Sparse Arrays for Direction of Arrival Estimation Under High Mutual Coupling,” *IEEE Transactions on Signal Processing*, vol. 72, pp. 4444–4460, 2024.

The reference work addresses the effect of mutual coupling in sparse arrays by designing array geometries with reduced or zero coarray weights at critical small lags. In particular, the paper proposes sparse arrays with constraints such as

$$w(1) = 0 \tag{1}$$

and

$$w(1) = w(2) = 0, \quad (2)$$

where $w(\ell)$ is the number of sensor pairs whose position difference equals lag ℓ .

The reference paper's central idea is:

Geometry-level design reduces mutual coupling sensitivity by eliminating or reducing sensor-pair contributions at small lags.

The paper also shows that many of the proposed arrays do not have a central ULA segment in the coarray. Instead, they use a one-sided contiguous ULA segment beginning at a positive lag such as $L_1 = 2$ or $L_1 = 3$. The maximum number of identifiable DOAs using the one-sided segment is tied to the segment length L and is typically expressed as

$$D_{\max} = \left\lfloor \frac{L}{2} \right\rfloor. \quad (3)$$

2.2 Identified Gap in the Reference Work

The reference work is primarily a geometry-design contribution. It focuses on reducing coupling-induced bias by controlling the array layout and the coarray weights at small lags. However, an uneven coarray weight distribution creates a separate statistical issue in finite-snapshot regimes.

For a sample covariance matrix estimated from M snapshots, a lag estimate $\hat{r}[\ell]$ is typically obtained by averaging all covariance entries whose sensor separation equals ℓ :

$$\hat{r}[\ell] = \frac{1}{w[\ell]} \sum_{n_i - n_j = \ell} [\hat{R}_x]_{i,j}. \quad (4)$$

A first-order variance proxy suggests that low-weight lags have higher estimation variance:

$$\text{Var}\{\hat{r}[\ell]\} \propto \frac{\sigma_n^2}{w[\ell]M}. \quad (5)$$

Thus, a geometry that helps mutual coupling can still produce noisy long-lag or low-weight coarray estimates when M is finite. This motivates a post-geometry statistical processing layer.

2.3 Correct Academic Positioning of This Project

The project should not initially claim to invent a new sparse-array geometry. The strongest and most defensible positioning is:

This work extends weight-constrained sparse-array DOA estimation by adding a post-geometry statistical denoising layer for finite-snapshot coarray correlations. The array geometry reduces mutual-coupling bias, while ALSS reduces variance in noisy lag estimates before virtual covariance reconstruction.

In compact form:

Layer	Mechanism	Primary Effect
Reference WCSA geometry	$w(1) = 0, w(1) = w(2) = 0$	Reduces coupling sensitivity / bias
Proposed ALSS processing	Lag-selective shrinkage	Reduces finite-snapshot variance
Combined framework	Geometry + statistical denoising	Improves robust Coarray MUSIC

3 Research Question and Hypothesis

3.1 Main Research Question

The primary research question is:

Can lag-aware statistical shrinkage improve Coarray MUSIC performance in weight-constrained sparse arrays without destroying the coupling-mitigation benefit of the original geometry?

More specifically:

Given a sparse array with uneven coarray weights $w[\ell]$, can low-confidence lag correlations be regularized before virtual covariance reconstruction in order to improve finite-snapshot DOA estimation?

3.2 Technical Hypothesis

The technical hypothesis is:

In weight-constrained sparse arrays, low-weight coarray lags have higher sample-estimation variance. Applying lag-selective shrinkage to these lags before Toeplitz covariance reconstruction can improve Coarray MUSIC robustness, especially under limited snapshots, low-to-moderate SNR, and mutual coupling.

3.3 Bias–Variance Interpretation

A useful conceptual decomposition is:

$$\text{MSE}\{\hat{\theta}\} = \text{Bias}^2\{\hat{\theta}\} + \text{Var}\{\hat{\theta}\}. \quad (6)$$

Within this framework:

- Mutual coupling introduces systematic steering-vector distortion and therefore contributes primarily to bias.
- Finite snapshots and uneven lag weights produce random estimation error and therefore contribute primarily to variance.
- Weight-constrained array geometry and ALSS can be complementary because they address different components of the error.

This interpretation should be used carefully. It is a guiding framework, not a substitute for rigorous numerical validation.

4 Proposed Technical Framework

4.1 Layer 1: Geometry and Coarray Analysis

The geometry layer computes and validates the following properties for each array:

- physical sensor positions $\{n_i\}_{i=1}^N$,

- difference coarray $\mathcal{D} = \{n_i - n_j\}$,
- coarray weights $w[\ell]$,
- one-sided and two-sided holes,
- largest contiguous one-sided segment $[L_1, L_2]$,
- segment length $L = L_2 - L_1 + 1$,
- maximum identifiable DOAs $D_{\max} = \lfloor L/2 \rfloor$.

The current repository already includes an abstract processing structure through `BaseArrayProcessor`. This should be preserved and cleaned because it provides a strong software architecture for reusable array analysis.

4.2 Layer 2: Signal Processing and DOA Estimation

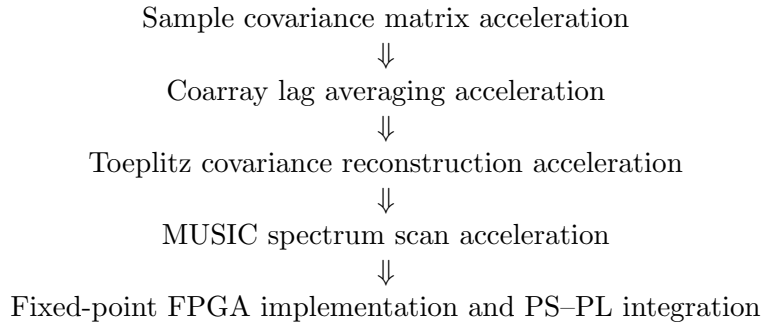
The signal-processing layer implements:

- narrowband far-field signal simulation,
- sample covariance matrix estimation,
- mutual coupling simulation,
- coarray lag averaging,
- virtual covariance reconstruction,
- Coarray MUSIC or one-sided coarray root-MUSIC,
- ALSS regularization,
- RMSE and resolution-rate evaluation.

4.3 Layer 3: FPGA-Ready Acceleration Path

The FPGA/HLS layer is a future extension that connects this project to hardware acceleration. The already completed SCM accelerator project demonstrates that covariance formation is a defensible FPGA kernel for DOA preprocessing.

The proposed hardware path is:



This hardware path should remain in the paper as future work until each kernel is implemented and verified.

5 ALSS Method Definition

5.1 Purpose of ALSS

ALSS is a coarray-domain denoising method applied after sample covariance estimation and lag averaging, but before virtual covariance reconstruction and subspace decomposition.

The intended processing chain is:

$$X \rightarrow \hat{R}_x \rightarrow \hat{r}[\ell] \rightarrow \hat{r}_{\text{ALSS}}[\ell] \rightarrow \hat{R}_v \rightarrow \text{Coarray MUSIC}. \quad (7)$$

5.2 General Shrinkage Form

The general shrinkage form is:

$$\hat{r}_{\text{ALSS}}[\ell] = \alpha[\ell]\hat{r}[\ell] + (1 - \alpha[\ell])r_{\text{prior}}[\ell], \quad (8)$$

where $\alpha[\ell] \in [0, 1]$ is a lag-dependent retention factor.

A conservative zero-prior version is:

$$r_{\text{prior}}[\ell] = 0 \quad (9)$$

for long or low-confidence lags.

5.3 Variance Proxy

The variance proxy used for shrinkage scheduling is:

$$\hat{V}[\ell] = \frac{\hat{\sigma}_n^2}{w[\ell]M}. \quad (10)$$

The exact implementation must be aligned with the paper. Two possible forms are acceptable, but only one should be selected and used consistently.

5.3.1 Option A: Retention-Factor ALSS

$$\alpha[\ell] = \frac{1}{1 + \tau\hat{V}[\ell]}. \quad (11)$$

Then Eq. (8) is used directly.

5.3.2 Option B: Variance-Ratio Shrinkage

A variance-ratio shrinkage form may instead define a shrinkage amount $\gamma[\ell]$ and use

$$\hat{r}_{\text{ALSS}}[\ell] = (1 - \gamma[\ell])\hat{r}[\ell]. \quad (12)$$

If the code uses this formulation, the paper must describe this formulation exactly. The repository must not contain a mismatch between the mathematical algorithm and the implementation.

5.4 Core-Lag Protection

Low lags with sufficiently high weights are protected:

$$\alpha[\ell] = 1, \quad \ell \leq L_0. \quad (13)$$

A simple definition is:

$$L_0 = \max\{\ell : w[\ell] \geq w_{\min}\}, \quad w_{\min} = 3. \quad (14)$$

This parameter may be refined later, but the first paper should use a simple and reproducible rule.

5.5 Hermitian Symmetry

After shrinkage, Hermitian symmetry must be enforced:

$$\hat{r}_{\text{ALSS}}[-\ell] = \hat{r}_{\text{ALSS}}^*[\ell]. \quad (15)$$

This is essential because the virtual covariance matrix must preserve Hermitian structure before MUSIC processing.

6 Repository Restructuring Plan

6.1 Target Repository Structure

The final repository should be organized as follows:

```
MIMO_GEOMETRY_ANALYSIS/  
|-- README.md  
|-- pyproject.toml  
|-- requirements.txt  
|-- references/  
| |-- references.bib  
| |-- reference_paper_summary.md  
|-- docs/  
| |-- technical_proposal_roadmap.tex  
| |-- paper_to_code_map.md  
| |-- reproduction_guide.md  
| |-- experiment_summary.md  
| |-- alss_algorithm_notes.md  
| |-- literature_positioning.md  
|-- geometry_processors/  
| |-- base/  
| | |-- base_array_processor.py  
| |-- reference_kulkarni/  
| | |-- z1_kulkarni.py  
| | |-- z4_kulkarni.py  
| | |-- z5_kulkarni.py  
| | |-- z6_kulkarni.py  
| |-- custom/  
| | |-- z5_custom.py  
| |-- validation/  
| | |-- validate_geometry_tables.py
```

```

|-- core/
| |-- radarpy/
| | |-- algorithms/
| | | |-- alss.py
| | | |-- coarray.py
| | | |-- coarray_music.py
| | |-- signal/
| | | |-- doa_sim_core.py
| | | |-- mutual_coupling.py
| | |-- metrics/
|-- scripts/
| |-- run_geometry_tables.py
| |-- run_alss_baseline_experiments.py
| |-- run_alss_mcm_experiments.py
| |-- run_snr_sweep.py
| |-- run_snapshot_sweep.py
| |-- run_all_paper_experiments.py
|-- tools/
| |-- plot_geometry_figures.py
| |-- plot_alss_results.py
| |-- export_paper_tables.py
|-- results/
| |-- raw/
| |-- processed/
| |-- figures/
| |-- manifests/
|-- papers/
| |-- alss_ieee_paper.tex
| |-- figures/
|-- tests/
| |-- test_geometry_processors.py
| |-- test_alss.py
| |-- test_coarray_music.py
| |-- test_reproducibility_smoke.py

```

6.2 Important Naming Rule

Reference geometries and custom geometries must be separated.

- If an implementation exactly follows the reference paper, name it with suffix `_kulkarni`.
- If an implementation is hand-tuned, perturbed, or modified, name it with suffix `_custom`.
- Do not call a custom geometry Z5 unless it is mathematically identical to the reference Z5 construction.

This rule is essential for academic integrity and reproducibility.

7 Work Packages

7.1 WP1: Literature and Source Alignment

Goal: Clarify what the reference paper contributed and where this project begins.

ID	Task	Deliverable
WP1.1	Summarize Kulkarni and Vaidyanathan’s WCSA contribution.	<code>docs/reference_paper_summary.md</code>
WP1.2	Identify exact definitions of Z1, Z4, Z5, Z6, and WCSA variants.	Geometry notes
WP1.3	Define the statistical estimation gap caused by uneven coarray weights.	<code>docs/literature_positioning.md</code>
WP1.4	Build a clean BibTeX file.	<code>references/references.bib</code>

7.2 WP2: Geometry Processor Cleanup

Goal: Make every array geometry reproducible, traceable, and academically defensible.

ID	Task	Deliverable
WP2.1	Preserve and clean the abstract base class architecture.	<code>base_array_processor.py</code>
WP2.2	Implement exact reference geometries from the Kulkarni paper.	<code>reference_kulkarni/*.py</code>
WP2.3	Move hand-tuned geometries to a custom folder.	<code>custom/*.py</code>
WP2.4	Validate $w(1)$, $w(2)$, $w(3)$, aperture, holes, and contiguous segment length.	<code>validate_geometry_tables.py</code>
WP2.5	Export geometry tables automatically.	<code>results/processed/geometry_summary.csv</code>

7.3 WP3: ALSS Algorithm Alignment

Goal: Ensure that the paper formula and code implementation are identical.

ID	Task	Deliverable
WP3.1	Select final ALSS formulation: retention-factor or variance-ratio shrinkage.	<code>docs/alss_algorithm_notes.md</code>
WP3.2	Update <code>alss.py</code> to match the selected formulation.	<code>core/radarpy/algorithms/alss.py</code>
WP3.3	Add tests for shrinkage behavior and Hermitian symmetry.	<code>tests/test_alss.py</code>
WP3.4	Verify that ALSS does not change protected core lags.	Unit test report

7.4 WP4: Reproducible Experiments

Goal: Regenerate all numerical claims from scripts.

ID	Experiment	Output
WP4.1	Geometry table reproduction.	<code>geometry_summary.csv</code>
WP4.2	Coarray weight plots.	<code>figures/coarray_weights_*.png</code>
WP4.3	Baseline Coarray MUSIC without ALSS.	<code>baseline_rmse.csv</code>
WP4.4	ALSS vs non-ALSS without MCM.	<code>alss_nomcm.csv</code>

WP4.5	ALSS vs non-ALSS with MCM.	<code>alss_mcm.csv</code>
WP4.6	SNR sweep.	<code>snr_sweep.csv</code>
WP4.7	Snapshot sweep.	<code>snapshot_sweep.csv</code>
WP4.8	Optional bias-variance decomposition.	<code>bias_variance.csv</code>

7.5 WP5: Paper-to-Code Map

Goal: Every table, figure, and major claim in the paper must be traceable to code.

Paper Item	Claim / Output	Script / Source	Status
Introduction	WCSA motivation and mutual coupling issue	Literature summary	Verified after source review
Table I	Array geometry specifications	<code>run_geometry_tables.py</code>	Pending regeneration
Algorithm 1	ALSS definition	<code>alss.py</code>	Needs formula-code alignment
Figure 1	Coarray weight distributions	<code>plot_geometry_figures.py</code>	Pending
Table II	ALSS without MCM	<code>run_alss_baseline_experiments.py</code>	Pending
Table III	ALSS with MCM	<code>run_alss_mcm_experiments.py</code>	Pending
Figure 2	SNR sweep	<code>run_snr_sweep.py</code>	Pending
Figure 3	Snapshot sweep	<code>run_snapshot_sweep.py</code>	Pending
Future Work	FPGA-ready acceleration path	SCM HLS project report and code	Conceptually verified

7.6 WP6: IEEE-Style Paper Rewrite

Goal: Rewrite the paper as one clean, defensible, reproducible IEEE-style article.

Recommended paper title:

Adaptive Lag-Selective Shrinkage for Robust Coarray MUSIC in Weight-Constrained Sparse Arrays

Recommended sections:

1. Introduction
2. Background: WCSA and Coarray MUSIC
3. Finite-Snapshot Variance Problem
4. Proposed ALSS Method
5. Experimental Framework
6. Results and Discussion
7. FPGA-Ready Acceleration Path
8. Conclusion and Future Work

Important recommendation: The first clean paper should focus on ALSS only. ALSS-II should be moved to future work or a separate extended paper until its claims are independently validated.

7.7 WP7: GitHub and Portfolio Presentation

Goal: Make the repository readable by professors, reviewers, and hiring managers.

The README should answer:

- What problem does the project solve?
- What is the reference paper?
- What is the new contribution?
- How are the figures and tables reproduced?
- How does the project connect to FPGA/HLS acceleration?

Avoid unverified phrases such as:

- “Production-ready”
- “100% test coverage”
- “1000-trial validation”
- “real-time deployment”

Use more accurate wording:

Research-grade reproducible Python framework for geometry-aware Coarray MUSIC experiments and ALSS regularization in weight-constrained sparse arrays.

8 Experiment Design

8.1 Baseline Signal Model

The narrowband array signal model is:

$$x[k] = A(\theta)s[k] + n[k], \quad (16)$$

where $A(\theta)$ is the array manifold matrix, $s[k]$ is the source vector, and $n[k]$ is additive white Gaussian noise.

The sample covariance matrix is:

$$\hat{R}_x = \frac{1}{M} \sum_{k=1}^M x[k]x^H[k]. \quad (17)$$

8.2 Mutual Coupling Model

For simulation, mutual coupling may be modeled as:

$$x_c[k] = CA(\theta)s[k] + n[k], \quad (18)$$

where C is a coupling matrix. A commonly used model is a decaying coupling matrix whose entries depend on sensor separation.

The exact mutual-coupling model must be documented clearly in every experiment, including:

- coupling strength,
- phase model, if complex coefficients are used,
- coupling bandwidth or truncation parameter,
- whether the same model as the reference paper is used.

8.3 Performance Metrics

Primary metrics:

- DOA RMSE,
- probability of resolution,
- failure rate or incorrect peak count,
- runtime,
- condition number of reconstructed virtual covariance.

Secondary metrics:

- bootstrap confidence intervals,
- paired statistical tests,
- bias–variance decomposition, if implemented carefully,
- sensitivity to snapshots and SNR.

8.4 Reproducibility Requirements

Every experiment must record:

- script name,
- command-line arguments,
- git commit hash,
- Python version,
- package versions,
- random seed policy,

- output CSV path,
- figure path,
- runtime.

A manifest file should be saved for each experiment:

```
results/manifests/
|-- alss_mcm_YYYYMMDD_HHMMSS_manifest.json
```

9 FPGA and Embedded-AI Development Path

9.1 Connection to the Existing SCM HLS Project

The previous FPGA project accelerated sample covariance matrix computation using Vitis HLS on a Zynq-7000 target. The project demonstrated the importance of array partitioning, loop unrolling, pipeline interleaving, and Hermitian symmetry. The strongest throughput design reduced latency from approximately 85,000 cycles to 10,610 cycles, while the area-oriented Hermitian design reduced DSP usage at the cost of higher latency.

This result supports the hardware feasibility of accelerating the front-end covariance stage of the DOA pipeline.

9.2 Candidate Hardware Kernels

Kernel	Description	FPGA Suitability
SCM formation	Compute $\hat{R}_x = \frac{1}{M} \sum x[k]x^H[k]$	Highly regular; already prototyped in HLS
Lag averaging	Map covariance entries to coarray lag estimates $\hat{r}[\ell]$	Moderate control; lookup-table friendly
ALSS shrinkage	Apply lag-dependent scalar shrinkage	Lightweight; mostly scalar operations
Toeplitz reconstruction	Build R_v from processed lags	Regular indexing; suitable for hardware memory mapping
MUSIC spectrum scan	Evaluate pseudospectrum over angle grid	High arithmetic density; good acceleration candidate
EVD/SVD	Subspace extraction	More complex; keep in software initially

9.3 Recommended Hardware Strategy

The immediate hardware strategy should not attempt a full MUSIC accelerator. Instead:

1. keep EVD/SVD in software,
2. accelerate covariance and coarray preprocessing kernels,
3. explore fixed-point reformulation,

4. use PS-PL partitioning on a Zynq/PYNQ platform,
5. document timing, latency, DSP, LUT, FF, and BRAM tradeoffs.

This is consistent with a defensible engineering roadmap and avoids overclaiming.

10 Course and Career Alignment

10.1 Summer Course: Embedded AI / Autonomous Robots

Connection:

- real-time sensing,
- radar perception,
- embedded signal processing,
- hardware-aware algorithm design.

Possible project angle:

Embedded radar perception pipeline with FPGA-ready Coarray DOA preprocessing.

10.2 Summer Course: Antenna Engineering

Connection:

- array geometry,
- mutual coupling,
- aperture constraints,
- element spacing and grating-lobe considerations,
- practical antenna-array layout.

Possible project angle:

Mutual-coupling sensitivity of weight-constrained sparse arrays from an antenna-engineering perspective.

10.3 Fall Course: Machine Learning

Connection:

- data-driven shrinkage,
- adaptive lag selection,
- learned priors for coarray denoising,
- model-assisted signal processing.

Possible extension:

Learning-assisted lag selection for Coarray MUSIC.

10.4 Fall Course: Python for Data Science

Connection:

- reproducible experiments,
- Monte Carlo simulation,
- confidence intervals,
- statistical visualization,
- data pipelines and experiment manifests.

Possible deliverable:

A fully reproducible Python research package with scripts, figures, tables, and experiment manifests.

11 Risk Register and Corrections

Risk	Problem	Mitigation
Formula-code mismatch	ALSS equation in paper may not match <code>alss.py</code> .	Select final formulation and align both paper and code.
Geometry ambiguity	Custom Z5 may be confused with reference Z5.	Separate <code>reference_kulkarni/</code> and <code>custom/</code> .
Overclaiming	Terms like “1000 trials”, “real-time”, or “production-ready” may be unsupported.	Only claim results regenerated from scripts.
ALSS-II complexity	Too many enhancements and strong numerical claims.	Move ALSS-II to future work or separate extended paper.
Mutual coupling interpretation	MCM may sometimes improve RMSE, making gap reduction ill-defined.	Use careful metrics and avoid invalid gap-reduction formulas when the denominator is negative.
Timing claims for FPGA	Python runtime and FPGA runtime can be confused.	Separate software experiment runtime from hardware acceleration estimates.

12 Immediate Action Plan

12.1 Priority 1: Freeze the Scientific Narrative

The first version of the project should focus on:

ALSS for finite-snapshot coarray denoising in weight-constrained sparse arrays.

Do not make the first clean paper about ALSS-II. ALSS-II can remain as a future enhancement.

12.2 Priority 2: Build the Paper-to-Code Map

Create:

```
docs/paper_to_code_map.md
```

This file becomes the master truth table of the project.

12.3 Priority 3: Correct Geometry Implementations

Implement exact reference geometries from the source paper and clearly separate custom geometries.

12.4 Priority 4: Regenerate All Results

Do not trust old tables or figures until they are regenerated from scripts.

12.5 Priority 5: Rewrite README and Paper

After scripts and results are aligned, rewrite:

```
README.md  
papers/alss_ieee_paper.tex
```

13 Expected Final Deliverables

1. Clean GitHub repository with documented structure.
2. Exact reference geometry processors.
3. Custom geometry processors clearly labeled.
4. ALSS implementation aligned with paper equations.
5. Reproducible experiment scripts.
6. CSV result tables and regenerated figures.
7. docs/paper_to_code_map.md.
8. docs/reproduction_guide.md.
9. docs/experiment_summary.md.
10. IEEE-style ALSS paper.
11. Hardware acceleration roadmap linked to the SCM HLS project.

14 Resume and Portfolio Positioning

A strong resume bullet after completion could be:

Developed a reproducible Python research framework for Coarray MUSIC DOA estimation in weight-constrained sparse arrays, including lag-selective shrinkage, mutual-coupling simulation, Monte Carlo validation, and FPGA-ready acceleration mapping.

A stronger hardware-focused bullet could be:

Built a DSP/FPGA research portfolio connecting sparse-array DOA estimation, coarray covariance processing, and Vitis HLS acceleration of covariance kernels for embedded radar applications.

A GitHub project tagline could be:

Research-grade framework for geometry-aware coarray denoising, robust DOA estimation, and FPGA-ready radar signal-processing acceleration.

15 Conclusion

This project has strong academic and professional potential. The core idea is scientifically defensible: weight-constrained sparse arrays reduce coupling sensitivity through geometry, but their uneven coarray weight distributions can create finite-snapshot variance problems. ALSS is a natural and defensible post-geometry regularization layer that addresses this gap.

The main weakness of the current project is not the idea itself; it is the lack of full alignment among the paper’s claims, code implementation, experimental scripts, and numerical results. The roadmap in this document is designed to solve that problem by rebuilding the project around traceability, reproducibility, and disciplined academic claims.

The final goal is to produce a professional research asset that supports coursework, publication development, GitHub credibility, and future FPGA/Embedded-AI career positioning.

References

- [1] P. Kulkarni and P. P. Vaidyanathan, “Weight-Constrained Sparse Arrays for Direction of Arrival Estimation Under High Mutual Coupling,” *IEEE Transactions on Signal Processing*, vol. 72, pp. 4444–4460, 2024.
- [2] P. Pal and P. P. Vaidyanathan, “Nested arrays: A novel approach to array processing with enhanced degrees of freedom,” *IEEE Transactions on Signal Processing*, vol. 58, no. 8, pp. 4167–4181, 2010.
- [3] P. P. Vaidyanathan and P. Pal, “Sparse sensing with co-prime samplers and arrays,” *IEEE Transactions on Signal Processing*, vol. 59, no. 2, pp. 573–586, 2011.
- [4] R. O. Schmidt, “Multiple emitter location and signal parameter estimation,” *IEEE Transactions on Antennas and Propagation*, vol. 34, no. 3, pp. 276–280, 1986.
- [5] H. L. Van Trees, *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory*. New York, NY, USA: Wiley, 2002.

- [6] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, “High-Level Synthesis for FPGAs: From Prototyping to Deployment,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, 2011.
- [7] H. Molhem, “Hardware-Aware Acceleration of Sample Covariance Matrix Computation via High-Level Synthesis for MIMO Radar,” EE6900 FPGA Accelerator Project, Kennesaw State University, 2026.
- [8] AMD/Xilinx, *Vitis High-Level Synthesis User Guide*, UG1399, 2025.